SC3020 Database System Principles

Group 8

Name	Matriculation Number	Individual Contributions
BHATI NANCY	U2222368G	20%
BRIAN TEY ZHI FONG	U2221714E	20%
BUI KHANH HUNG	U2322190J	20%
NYIM BOEY	U2320429D	20%
SAM CHERN KAI, CHESTER	U2322722J	20%

GENERATIVE AI TOOLS

Our group declares that no AI tools have been used in the report.

Section 1: Executing the file

Launching Steps

Windows OS:

1. Install cmake from cmake.org
2. Launch Directory: cd Project_1
3. Remove all past build directory: rm build
4. Remove all files inside ./bin folder
5. Create new build directory: mkdir build
6. Generate Makefiles: cmake -S . -B build -G "Visual Studio 17 2022" -A x64
7. Build File: cmake --build build --config Release

Mac OS:

1. Install cmake from cmake.org
2. Launch Directory: cd Project_1
3. Remove all past build directory: rm -rf build
4. Remove all files inside./bin folder
5. Create new build directory: mkdir build
6. Generate Makefiles: cmake -S . -B build
7. Build File: cmake --build build --config Release

Output:

1. Task 1 + Task 2 + Task 3:
   .\build\Release\sc3020_task1.exe --load .\data\games.txt --db .\bin\games.db --build-index --delete --delete-threshold 0.9

2. Optional: For point query
   .\build\Release\sc3020_task1.exe --load .\data\games.txt --db .\bin\games.db --build-index --search 0.952

3. Optional: For range query
   .\build\Release\sc3020_task1.exe --load .\data\games.txt --db .\bin\games.db --build-index --range 0.8 1.0

Note: To run each output, build folder and files in ./bin folder must be deleted.

If any errors are encountered:

1. rmdir build
2. input “A” to delete everything
3. delete files in ./bin folder

Output File Location:

- ➔ ./build folder populated.
- ➔ ./build/Release contains sc3020_task1.exe
- ➔ ./bin contains games.db, results_task123.txt

Section 2: Storage Component (Task 1)

2.1 Structure Used

We used a compact 39-bytes struct defined in record.hpp. It includes:

```
// Record Struct
struct Game {
    Date     game_date_est;        // 4 bytes
    char     team_id_home[10];     // VARCHAR(10) stored as exactly 10 bytes
    int32_t  pts_home;             // int32
    float    fg_pct_home;          // float32
    float    ft_pct_home;          // float32
    float    fg3_pct_home;         // float32
    int32_t  ast_home;             // int32
    int32_t  reb_home;             // int32
    uint8_t  home_team_wins;       // boolean (0/1)
};
```

Data Fields	Data Type	Size(Bytes)
GAME_DATE_EST	Date	4
TEAM_ID_home	Varchar(10)	10
PTS_home	int32	4
FG_PCT_home	float32	4
FT_PCT_home	float32	4
FG3_PCT_home	float32	4
AST_home	int32	4
REB_home	int32	4
HOME_TEAM_WINS	boolean (0/1)	1

Total bytes: 39 bytes

Supporting structs:

1. RID: It identifies a record's physical location i.e. blockId (page number), slotId (slot within the page) and pad (padding to make size 8 bytes)

2. SlotEntry: It keeps track of the offsets and sizes of each record stored on the page, as well as flags to indicate if a slot is used or marked deleted.
3. PageHeader: It contains metadata about each page like page ID, number of slots, directory end, and free space end.

2.2 How the Page System Works

Each page is 4096 bytes which includes:

- A PageHeader struct at the beginning (fixed size)
- A slot directory growing downwards (one SlotEntry per inserted record)
- Record data growing upwards from the end of the page

When a record is inserted:

- A SlotEntry is appended to the slot directory
- The actual record is copied to the free space from the back of the page
- The PageHeader is updated to reflect new dirEnd and freeEnd

```
//Initialise new page
void HeapFile::initPage(uint32_t pageId) const {
    std::vector<char> buf(pageSize_, 0);
    PageHeader hdr;
    hdr.pageId = pageId;
    hdr.slotCount = 0;
    hdr.dirEnd = sizeof(PageHeader);
    hdr.freeEnd = static_cast<uint16_t>(pageSize_);
    std::memcpy(buf.data(), &hdr, sizeof(PageHeader));
    disk_.writePage(pageId, buf.data());
}
```

2.3 How the File is Parsed

The file is parsed using the Loader class:

- On construction, it opens the input file (.tsv, .csv, .txt) and reads the header.
- It automatically detects the delimiter (\t or ,)
- The header line is split into column names, and relevant column indices are stored
- Each subsequent line is read and split into columns
- Each field is trimmed and converted to the appropriate type (int, float, etc.)
- If parsing succeeds, a Game struct is populated and returned

Any malformed or incomplete rows are skipped, ensuring robustness.

```cpp
Loader::Loader(const std::string& path) {
    in_.open(path);
    if (!in_) throw std::runtime_error("Loader: cannot open file: " + path);

    // Read header
    std::string header;
    if (!std::getline(in_, header)) throw std::runtime_error("Loader: empty file");

    //delimiter
    sep_ = (header.find('\t') != std::string::npos) ? '\t'
         : (header.find(',')  != std::string::npos) ? ','
                                                    : '\t';

    auto cols = split_by(header, sep_);
    for (auto& c : cols) c = trim(c);

    auto find_col = [&](const std::string& name) -> int {
        for (size_t i = 0; i < cols.size(); ++i) {
            if (cols[i] == name) return static_cast<int>(i);
        }
        return -1;
    };

    col_game_date_est = find_col("GAME_DATE_EST");
    col_team_id_home  = find_col("TEAM_ID_home");
    col_pts_home      = find_col("PTS_home");
    col_fg_pct_home   = find_col("FG_PCT_home");
    col_ft_pct_home   = find_col("FT_PCT_home");
    col_fg3_pct_home  = find_col("FG3_PCT_home");
    col_ast_home      = find_col("AST_home");
    col_reb_home      = find_col("REB_home");
    col_home_team_wins= find_col("HOME_TEAM_WINS");

    if (col_game_date_est < 0 || col_team_id_home < 0 || col_pts_home < 0 ||
        col_fg_pct_home < 0 || col_ft_pct_home < 0 || col_fg3_pct_home < 0 ||
        col_ast_home < 0 || col_reb_home < 0 || col_home_team_wins < 0) {
        throw std::runtime_error("Loader: required column missing in header (detected sep="
                                 + std::string(sep_ == '\t' ? "\\t" : ",") + ")");
    }
}
```

2.4 Data Insertion into Pages During Parsing

In the main.cpp file, data loading works like this:

- For each parsed Game record from Loader, heap.insert(game) is called
- HeapFile::insert():
    - Scans existing pages to check if there's enough space
    - If no page has room, a new page is allocated and initialized
    - Space is verified using ensureSpace(), which checks whether there's enough space to insert a new record
- Record is copied to the correct offset in the page's free space
- A SlotEntry is written into the slot directory
- Page header is updated

The result is a binary file (games.db) consisting of slotted pages, with compact and efficient storage of all parsed records

```cpp
//Inserts a new record
HeapFile::InsertResult HeapFile::insert(const Game& g) {
    uint32_t pages = static_cast<uint32_t>(numBlocks());
    if (pages == 0) {
        uint32_t pid = disk_.allocatePage();
        initPage(pid);
        pages = 1;
    }
    uint32_t target = pages - 1;
    if (!ensureSpace(target, sizeof(Game))) {
        target = disk_.allocatePage();
        initPage(target);
    }
    std::vector<char> buf(pageSize_);
    disk_.readPage(target, buf.data());
    PageHeader hdr{};
    std::memcpy(&hdr, buf.data(), sizeof(PageHeader));

    const uint16_t recLen = static_cast<uint16_t>(sizeof(Game));
    const uint16_t recOff = static_cast<uint16_t>(hdr.freeEnd - recLen);
    std::memcpy(buf.data() + recOff, &g, sizeof(Game));

    SlotEntry se{};
    se.offset = recOff;
    se.len = recLen;
    se.used = 1;
    se.deleted = 0;

    uint16_t slotId = hdr.slotCount;
    size_t slotOff = sizeof(PageHeader) + static_cast<size_t>(slotId) * sizeof(SlotEntry);
    std::memcpy(buf.data() + slotOff, &se, sizeof(SlotEntry));

    hdr.slotCount = static_cast<uint16_t>(hdr.slotCount + 1);
    hdr.dirEnd = static_cast<uint16_t>(hdr.dirEnd + sizeof(SlotEntry));
    hdr.freeEnd = recOff;
    std::memcpy(buf.data(), &hdr, sizeof(PageHeader));

    disk_.writePage(target, buf.data());

    InsertResult res;
    res.pageId = target;
    res.rid = { target, slotId, 0 };
    res.newPageAllocated = false;
    return res;
}
```

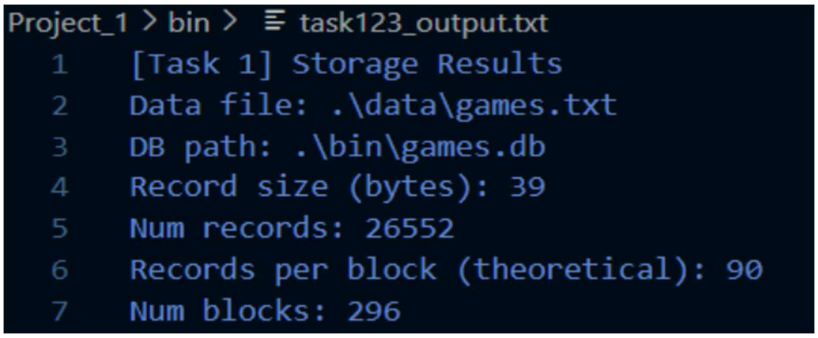
2.5 How to Determine if Page is Full

To determine if a page is full, the system checks whether there is enough space to insert both a record (fixed size: 39 bytes) or a SlotEntry (metadata about the record). In memory, each page is organized with Header and SlotEntries growing downwards and Records growing upwards from the bottom. If these two regions collide, the page is full. This check is done using ensureSpace() function.

```cpp
bool HeapFile::ensureSpace(uint32_t pageId, size_t recLen) const {
    std::vector<char> buf(pageSize_);
    disk_.readPage(pageId, buf.data());
    PageHeader hdr{};
    std::memcpy(&hdr, buf.data(), sizeof(PageHeader));
    uint16_t newDirEnd = static_cast<uint16_t>(hdr.dirEnd + sizeof(SlotEntry));
    if (newDirEnd > hdr.freeEnd) return false;
    if (hdr.freeEnd < recLen + newDirEnd) return false;
    return true;
}
```

2.6 Results

Here is the output of task 1:



There are a total of 26552 records. The block size is 4096 bytes. With record size 39 bytes, Slot entry size 6 bytes and Pageheader size 8 bytes, there will $\lfloor \frac{4096-8}{39+6} \rfloor = \lfloor \frac{4088}{45} \rfloor = 90$ records per block. Therefore, to store all records in the database, we will need $\lceil \frac{26552}{90} \rceil$ = 296 blocks.

Section 3: Building B+ Tree (Task 2)

3.1 Structure Used

To generate the structure to display the relevant information to be sorted according to ***FT_PCT_home*****,** we adopted a B+ tree. It allows for the information to be efficiently searched, inserted and retrieved, while ensuring that all records are stored at the leaf level for fast range queries.

In the B+ tree, there are several attributes to take note of, which is being highlighted in Table 3.1.1.

Attributes	**Definition**
Keys	Shows the sorted list of *FT_PCT_home* in ascending order.
Pointers	To point to the correct records / next leaf node.
n	Value of *n* to determine the number of keys and pointers.

Table 3.1.1: Illustration of the Key attributes in a B+ Tree.

From the first experiment, we can observe that there are **371** distinct *FT_PCT_home*. Based on this value, our team has decided to fix our value *n* to be at **30** instead. This implies that the maximum number of keys per node will be 30. Each internal node will also be able to direct query up to 31 subtrees while storing 30 distinct ***FT_PCT_home*** values.

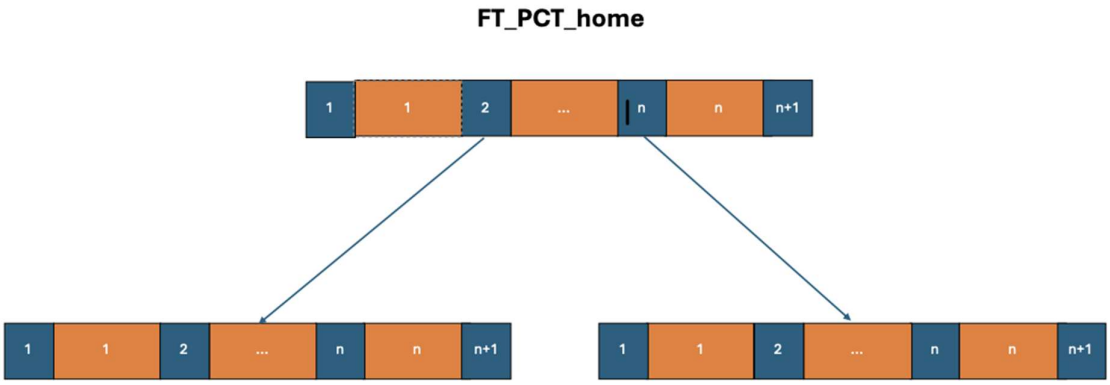


Figure 3.1.1: This figure illustrates how the B+ Tree will look like.

From the above Figure 3.1.1 illustrated, several key features can be observed.

3.1.1 Feature 1: Records are kept in leaf level

A key feature of a B+ Tree allows all records to only be kept at the leaf level. This helps to ensure uniform search paths and all queries to be straightforward.

As for the internal nodes, they are focussed on storing keys and corresponding routing pointers.

3.1.2 Feature 2: Balanced Structure

B+ Tree also helps to ensure a well balanced structure where all leaves appear to be the same depth.

As a result, we can see that search, insert and delete operations are predictable ***O(log n)*** complexity.

3.1.3 Feature 3: Support for duplicates

Multiple ***TEAM_ID_home*** can share the same ***FT_PCT_home***. In our code implementation, each leaf entry stores a vector of RIDs to capture all rows with the same ***FT_PCT_home*** value.

3.1.4 Feature 4: Dynamic Growth and Shrinkage

Another big key feature of a B+ Tree allows for nodes to be able to merge and split when the nodes are full and under-utilized respectively. As such, indexes in B+ Tree can adjust dynamically to insert and delete without a full reorganization.

3.2 Bulk Loading

In the lecture slides, we understand that there are mainly two methods in order to create a B+ tree using the existing collection of data.

Method 1: Repeatedly inserting data

Method 2: Bulk Loading

It is observed and understood that Method 2 is much more efficient and less time consuming compared to Method 1. In Method 1, inserting one key at a time will result in repeatedly splitting nodes and rebalancing, which will recursively call for many iterations of checking, which is often an inefficient method.

On the other hand, Method 2 applies a bottom-up bulk-loading algorithm that achieves a compact tree with optimal fill. The following steps are adopted to ensure that bulk loading of data is done successfully.

More will be shared about the steps to achieve this fill.

Steps	Description
1	We first group all the data with duplicate keys *FT_PCT_home*. This ensures that all ***(FT_PCT_home, RID)*** are combined into a single *(key, vector<RID>)* entry. By doing this, we reduce the number of indexed entries from **26,552** to **371** distinct values.
2	Next, we have to pack the entries so that they can form the leaf nodes. By using the predefined value of *n* to be 30, we can deduce that 371 keys will require minimally 13 leaf blocks.
3	With all the leaf blocks, they can be grouped to form the parent node. Each parent node will be filled with *(n+1)* children. This process will be iteratively performed until we can finally have 1 root node remaining.

From the data we have on hand, we can understand that the 14 leaf blocks can be grouped to form a single parent node as only 14 pointers will be required. As such, we can observe that our B+ tree will only be 2 levels tall after mass bulk loading of data as illustrated in Figure 3.2.1.

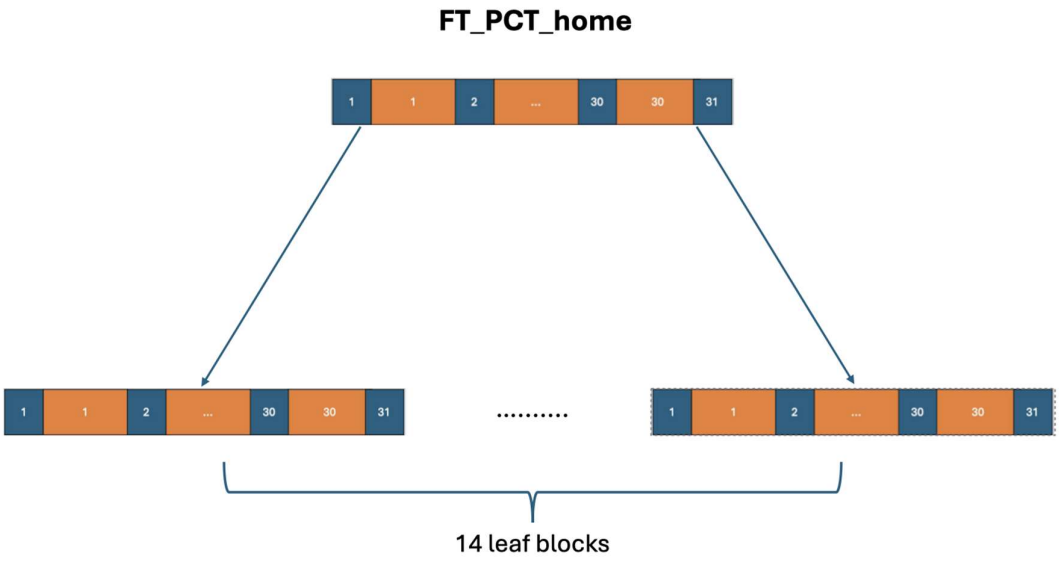


Figure 3.2.1: Illustration of the B+ Tree

3.3 Insertion into a Non-Full Node

In order to insert new data into the B+ tree, we first need to check whether the root node is full. This is necessary as if the root is already full and we descend without splitting, a split at the leaf node could force a key to be promoted into a root that has no room. This requires backtracking and restructuring which complicates the process. Thus, it is a key rule for B+ tree insertion where the algorithm works top-down; it ensures that the root is never full before insertion begins, and further splits full children nodes during descent. This guarantees that the leaf node where the new key is to be inserted always has space available and avoids upward recursive splits.

Assuming that the leaf node is non-full when inserting a key, we find the correct position of the key by using binary search to find the first position where the key can be inserted without breaking sorted order. If the key already exists, the new *RID* is appended to that key's bucket. Otherwise, the key and its ***RID*** are inserted in sorted order into the leaf node.

3.4 Handling Overflow in Insertion

When inserting into a B+ tree, if the target node is already full, meaning it contains the maximum number of keys equal to the tree's order n, an overflow occurs. In general, we divide the node into two and promote a separator key to the parent node. At each internal node, before descending, the algorithm checks if the child is full. If so, it splits the child and then descends. This ensures we never recurse into a full node. This runs recursively until we reach the leaf node, which is guaranteed to be non-full as we would have already split it. The key can then be safely inserted. Depending on what type of node is full, we will execute different steps as shown in the following elaboration.

3.4.1 Root Node

Since a root node is the only node without a parent, its overflows must be handled specially.

When the root overflows:

1. A new root node is created.
2. The old root node is split into two child nodes.
3. The separator key is inserted into the new root node.

This increases the tree's height by 1 and ensures that the B+ tree remains balanced.

3.4.2 Internal Node

Internal nodes are nodes between the first and last levels of the tree.

Assuming that the parent node is non-full, if an internal node overflows:

1. The node is split around its median key.
2. The keys left of the median stay in the original node and the keys to the right of the median are passed to a new internal node.
3. The child pointers are redistributed accordingly.
4. The median key itself is promoted into the parent node.
5. The parent will also have its pointers redistributed accordingly so that the new internal node is responsible for the right half of the original children.

3.4.3 Leaf Node

Leaf nodes are at the bottom-most layer of the tree, where keys are to be inserted directly.

Assuming that the parent node is non-full, if a leaf node overflows:

1. It is split into two leaf nodes, with roughly half the keys in each.
2. The first key of the new right leaf is promoted into the parent node as the separator.
3. The leaf-level "next" pointer is updated to maintain the linked list of leaves. This ensures that range queries still function correctly across the leaf level.

3.5 Results

Based on the data obtained from the first task, we can observe that there are **371** distinct keys. As such, we ran a simulation to deduce the best tree with different values of *n*. The results can be illustrated by the following table 3.5.1.

***n* value**	3	7	10	15	20	25	30
B+ Tree Level	5	3	3	3	2	2	2
Num of Nodes	165	61	42	28	20	16	14

Table 3.5.1: Corresponding Levels and Number of nodes for each respective *n* value.

With the above results shown, we ultimately decided on setting the value of *n* to be the value 30. Below are the results we obtained for task 2 using *n=30*.

B+ Tree Levels :	2
Number of Nodes:	14
Root Key Values:	[0.433, 0.52, 0.567, 0.614, 0.653, 0.703, 0.741, 0.784, 0.828, 0.871, 0.915, 0.966]

Table 3.5.2: Results from choosing ***n*** to be 30.

Moreover, here are some benefits from using setting the value of *n* to be *30*.

Advantage 1: Shallow Tree Height of 2 results in lower I/O cost and faster searches with just a level of 2.

Advantage 2: Compact Index size of 14 allows for minimal storage overhead

Advantage 3: Scalability margin is available as the root can support up to 31 children and only 13 are used currently, which shows that there is plenty of room to expand.

Section 4: Executing Query (Task 3)

4.1 Structure Used

4.1.1 Purpose

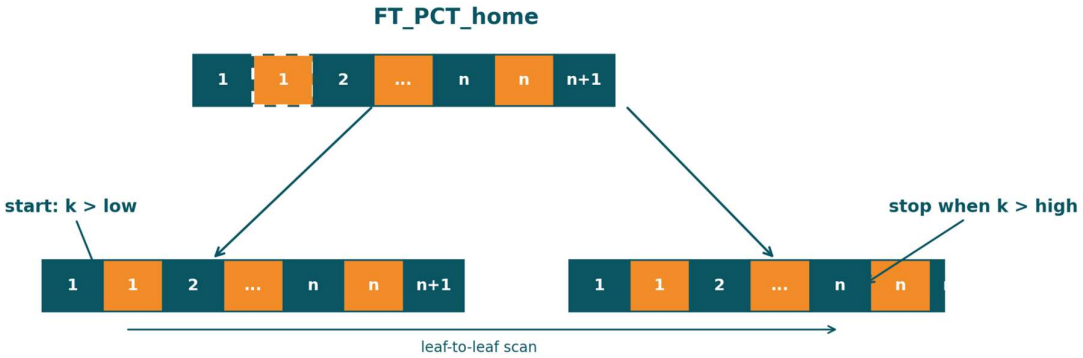
- Support queries of the form ***FT_PCT_home > T*** and return matching rows, then delete them while keeping the heap file and B+-tree consistent.

4.1.2 Core data structures

- **Storage & IDs:** We still operate on the same slotted-page heap file; qualifying rows are addressed by ***RID = {blockID, slotID}***.
- **Index:** In-memory B+-tree over key ***FT_PCT_home (float),*** order ***n = 30***, leaves linked via ***next***. Duplicate keys map to ***vector<RID>*** in each leaf slot.
- **Range retrieval handle:** ***RangeCollect(low, high)*** returns ***(key, RID)*** pairs and ***indexNodesAccessed*** (count of index nodes visited during retrieval).
- **Instrumentation:**
    - index nodes accessed (retrieval)
    - distinct data blocks touched (when deletions are applied)
    - number deleted & average deleted key
    - index retrieval time vs brute-force time/blocks.

4.2 How are Matches Being Found

4.2.1 Index Range Retrieval



- **Descend to the start leaf.** From the root, binary-search separator keys using the lower bound ***low*** to reach the first candidate leaf. Count each internal node visited and the first leaf in ***indexNodesAccessed***.
- **Scan along the leaf chain.** Follow leaf → next, visiting keys in order:
  - **Select:** emit every ***(k, RID)*** where k > low && k < high.
  - **Early stop:** once ***k*** is greater than ***high***, terminate (skipped when `high = +∞`).
  - **Duplicates:** for a qualifying key ***k***, output all RIDs in its RID vector.
  - **Accounting:** increment ***indexNodesAccessed*** for each subsequent leaf we step into.
- **T-only queries:** implement strict > T by calling `RangeCollect(T, +∞)`.
- **Cost:** O(h+L) — where ***h*** `is the tree height (≈ 2 with n=30) and` ***L*** is the number of leaf nodes scanned until early stop.

4.2.2 Retrieval outputs & counters

- **Output shape:** a stream (or vector) of (key, RID) pairs in non-decreasing key order.

```
const float low  = args.deleteThreshold;
const float high = std::numeric_limits<float>::infinity();

auto time0 = std::chrono::high_resolution_clock::now();
auto rangeCollectIndex = index.rangeCollect(low, high);
auto time1 = std::chrono::high_resolution_clock::now();
t3_retrievalMillis = std::chrono::duration<double, std::milli>(time1 - time0).count();
t3_indexNodesAccessed = rangeCollectIndex.indexNodesAccessed;
```

- **Instrumentation (retrieval-only):** record ***indexNodesAccessed*** and the index retrieval time.

```
auto btime0 = std::chrono::high_resolution_clock::now();
size_t bruteCnt = 0; double bruteSum = 0.0;
heap.scan([&](uint32_t, uint16_t, const Game& g, bool tomb) {
    if (!tomb && g.ft_pct_home > args.deleteThreshold) {
        bruteCnt++; bruteSum += g.ft_pct_home;
    }
});
auto btime1 = std::chrono::high_resolution_clock::now();
t3_bruteforceMillis = std::chrono::duration<double, std::milli>(btime1 - btime0).count();
t3_bruteforceBlocks = heap.numBlocks();
(void)bruteCnt; (void)bruteSum;
```

- **Baseline comparison:** optionally run a full heap scan with the same predicate to compare brute-force time/blocks

4.3 Deletion and Erasure

4.3.1 Deletion in Heap File

As deletion is done in a heap file, there is no physical modification on the data. Instead, a '**deleted**' flag is used to indicate that the record has been logically deleted.

- Each record resides in a page, and its location is tracked by a slot entry containing offset, length and status bits.
- When a deletion is requested, the code invokes ***HeapFile::markDeleted(const RID& rid)***.
- The function then reads the slot entry for the given RID and sets the deleted flag to 1.
- The data bytes of the record remain in the page but are no longer considered valid.

This design avoids expensive in-place compaction of data and ensures that other references to the page remain valid. It is fast, requiring only a slot directory update and avoids rewriting the page structure.

4.3.2 Deletion in B+ Tree

The B+ tree also enforces the deletions to maintain query correctedness. This is being handled by ***BPlusTree::erase()*** and its helper functions as below.

- ***BPlusTree::eraseRecursive( ),***
- ***BPlusTree::borrowFromLeft( ),***
- ***BPlusTree::borrowFromRight( ),***

- ***BPlusTree::mergeNodes( )***

Process:

1. Locate the key:
    a. The algorithm will go down the tree using key comparisons until the relevant leaf node has been reached
    b. If the key is present, the associated vector of RIDs is accessed
2. Remove the RID:
    a. Within the lead, the specific RID is removed from the vector
    b. If there exists other RIDs under the same key, the key stays in the leaf
3. Remove the key if empty:
    a. If the RID vector becomes empty, the key is erased from the leaf node entirely
4. Rebalancing the B+ Tree:
5. If the deletion cause the node to fall below the minimum occupancy, the algorithm restores balance by either
    a. Borrowing from a sibling. The algorithm checks the left sibling first then the right sibling and then updates the parent separator.
    b. Merging with a sibling. When borrowing from a sibling is not possible, it merges with a sibling node and the parent's node separator key is removed.
6. Root adjustment:
    a. If the root becomes empty after a merge, its single child becomes the new root, and reduces the tree height as a result.

4.4 Results

4.4.1 Task 3 Statistics

Index Nodes Accessed	4
Data Blocks Accessed	293
Number of Games Deleted	1778
Avg FT_PCT_home (deleted)	0.939643

Retrieval Time (ms)	0.0631
Brute-force Blocks Accessed	296
Brute-force Time (ms)	4.7587

Explanation:

- B+ Tree method scanned 3 blocks less than brute force method, which meant that for ***FT_PCT_home*** > 0.9, there are 293 blocks that contain ***FT_PCT_home*** with values larger than 0.9 and 2 blocks without any ***FT_PCT_home*** with values larger than 0.9.
- Brute force takes much longer as it has to scan through the heap file one by one while B+ tree makes use of the index to speed up access time

4.4.2 Updated B+ Tree Stats (After Deletions)

Levels	2
Nodes	12
Root Keys	0.433, 0.52, 0.567, 0.614, 0.653, 0.703, 0.741, 0.784, 0.828, 0.871

Explanation:

- The number of levels are the same, which means that there are insufficient deletions that will cause the level of the tree to decrease.
- There are 2 nodes less as compared to non-deleted dataset.
- After deletion, root keys that are larger than 0.9 are removed due to rebalancing. This also corresponds to the 2 nodes deleted.